매트릭스 패킹 알고리즘

개요

Irys 체인의 매트릭스 패킹 알고리즘은 2D 해시 패킹 기법을 사용하여 데이터 청크를 암호화적으로 안전한 방식으로 변환합니다. 이 알고리즘은 채굴자의 주소, 파티션 해시, 청크 오프셋을 기반으로 고유한 엔트로피를 생성하여 데이터를 패킹합니다.

수학적 원리

기본 개념

매트릭스 패킹은 다음과 같은 수학적 원리를 기반으로 합니다:

1. **시드 해시 생성**: `H(address || partition_hash || chain_id || offset)`
2. **순차적 해싱**: 초기 엔트로피 매트릭스 생성
3. **2D 해시 패킹**: 매트릭스 내 요소들의 상호 의존적 해싱
4. **XOR 연산**: 원본 데이터와 엔트로피의 XOR 결합

알고리즘 상수

```
#define PACKING_HASH_SIZE 32 // SHA-256 해시 크기
#define HASH_ITERATIONS_PER_BLOCK 8192 // 블록당 해시 반복 횟수
#define DATA_CHUNK_SIZE (HASH_ITERATIONS_PER_BLOCK * PACKING_HASH_SIZE) // 262,
```

패킹 프로세스

1단계: 시드 해시 계산

```
pub fn compute_seed_hash(
address: Address,
offset: std::ffi::c_ulong,
hash: [u8; SHA_HASH_SIZE],
irys_chain_id: u64,
) -> [u8; SHA_HASH_SIZE] {
let mut hasher = sha::Sha256::new();
let address_buffer: [u8; 20] = address.0.into();
// 입력 데이터 연결
hasher.update(&address_buffer); // 채굴자 주소 (20바이트)
hasher.update(&hash); // 파티션 해시 (32바이트)
hasher.update(&irys_chain_id.to_le_bytes()); // 체인 ID (8바이트)
hasher.update(&offset.to_le_bytes()); // 청크 오프셋 (8바이트)
hasher.finish()
}
```

시드 해시의 역할

- **고유성 보장**: 각 청크마다 고유한 시드 생성
- **결정론적**: 동일한 입력에 대해 항상 동일한 결과

- **암호학적 안전성**: SHA-256의 보안 특성 활용

2단계: 순차적 해싱 (Phase 1)

```
pub fn compute_entropy_chunk(
mining_address: Address,
chunk_offset: std::ffi::c_ulong,
partition_hash: [u8; SHA_HASH_SIZE],
iterations: u32,
chunk_size: usize,
out_entropy_chunk: &mut Vec<u8>,
irys_chain_id: u64,
) {
// 시드 해시 계산
let mut previous_segment = compute_seed_hash(
mining_address,
chunk_offset,
partition_hash,
irys_chain_id
);
out_entropy_chunk.clear();
// Phase 1: 순차적 해싱으로 초기 엔트로피 매트릭스 생성
for _i in 0..(chunk_size / SHA_HASH_SIZE) {
previous_segment = sha::sha256(&previous_segment);
```

```
for j in 0..SHA_HASH_SIZE {
out_entropy_chunk.push(previous_segment[j]);
}
}
// Phase 2는 다음 단계에서 설명
}
```

순차적 해싱의 특징

- **체인 구조**: 각 해시가 이전 해시에 의존
- **확산 효과**: 작은 입력 변화가 전체 결과에 영향
- **효율성**: 단순한 순차 처리로 빠른 계산

3단계: 2D 해시 패킹 (Phase 2)

```
// Phase 2: 2D 해시 패킹
let mut hash_count = chunk_size / SHA_HASH_SIZE;
debug_assert_ne!(hash_count, 0, "0 2D packing iterations - is your chunk_size <

while hash_count < iterations as usize {
let i = (hash_count % (chunk_size / SHA_HASH_SIZE)) * SHA_HASH_SIZE;
let mut hasher = sha::Sha256::new();
// 이전 세그먼트 선택
if i == 0 {
```

```
hasher.update(&out_entropy_chunk[chunk_size - SHA_HASH_SIZE..]);
} else {
hasher.update(&out_entropy_chunk[i - SHA_HASH_SIZE..i]);
}
// 현재 세그먼트 추가
hasher.update(&out_entropy_chunk[i..i + SHA_HASH_SIZE]);
// 새로운 해시로 현재 위치 업데이트
let  hash = hasher.finish();
out_entropy_chunk[i..i + SHA_HASH_SIZE].copy_from_slice(&hash);
hash_count += 1;
}
```

2D 패킹의 수학적 모델

매트릭스를 M[i][j]로 표현할 때:

```
M[i] = H(M[i-1] || M[i]) for i > 0
M[0] = H(M[n-1] || M[0]) for wraparound
```

여기서:

- `H()` : SHA-256 해시 함수
- `||` : 바이트 연결 연산
- `n` : 총 세그먼트 수

4단계: 데이터 패킹 (XOR 연산)

```rust
#[inline]
pub fn packing_xor_vec_u8(mut entropy: Vec<u8>, data: &[u8]) -> Vec<u8> {
debug_assert!(data.len() <= entropy.len());
for i in 0..data.len() {
entropy[i] = entropy[i].bitxor(data[i]);
}
entropy
}
```

XOR 패킹의 특성

- **가역성**: `A ⊕ B ⊕ B = A`
- **균등 분포**: 엔트로피가 균등하면 결과도 균등
- **효율성**: 비트 단위 연산으로 매우 빠름

언패킹 프로세스

엔트로피 재생성

```rust
pub fn unpack(
packed_chunk: &PackedChunk,
entropy_packing_iterations: u32,
chunk_size: usize,
irys_chain_id: u64,
```

```
) -> UnpackedChunk {
// 동일한 파라미터로 엔트로피 재생성
let mut entropy: Vec<u8> = Vec::with_capacity(chunk_size);
capacity_single::compute_entropy_chunk(
packed_chunk.packing_address,
*packed_chunk.partition_offset as u64,
packed_chunk.partition_hash.0,
entropy_packing_iterations,
chunk_size,
&mut entropy,
irys_chain_id,
);

// XOR 연산으로 원본 데이터 복원
let unpacked_data = unpack_with_entropy(packed_chunk, entropy, chunk_size);

UnpackedChunk {
data_root: packed_chunk.data_root,
data_size: packed_chunk.data_size,
data_path: packed_chunk.data_path.clone(),
bytes: Base64(unpacked_data),
tx_offset: packed_chunk.tx_offset,
}
```

```
}
```

크기 조정 및 트리밍

```
pub fn unpack_with_entropy(
packed_chunk: &PackedChunk,
entropy: Vec<u8>,
chunk_size: usize,
) -> Vec<u8> {
let chunk_size_u64: u64 = chunk_size.try_into().unwrap();
let mut unpacked_data = packing_xor_vec_u8(entropy, &(packed_chunk.bytes.0));


let data_size = packed_chunk.data_size;
let num_chunks_in_tx = data_size.div_ceil(chunk_size_u64);
if num_chunks_in_tx == 0 {
return unpacked_data;
}
// 마지막 청크인 경우 크기 조정
if (*packed_chunk.tx_offset as u64) == num_chunks_in_tx - 1 {
let trailing_bytes = data_size % chunk_size_u64;
if trailing_bytes != 0 {
unpacked_data.truncate(trailing_bytes.try_into().unwrap());
}
```

```
}

unpacked_data

}
```

패킹 효율성 분석

시간 복잡도

- **시드 해시**: O(1) - 상수 시간
- **순차적 해싱**: O(n) - 청크 크기에 비례
- **2D 패킹**: O(k) - 반복 횟수에 비례
- **XOR 연산**: O(n) - 데이터 크기에 비례

전체 시간 복잡도: **O(n + k)**

공간 복잡도

- **엔트로피 버퍼**: O(n) - 청크 크기
- **임시 해시**: O(1) - 32바이트 고정
- **출력 버퍼**: O(n) - 청크 크기

전체 공간 복잡도: **O(n)**

성능 특성

CPU 구현

```
// 벤치마크 결과 (256KB 청크 기준)

// - 엔트로피 생성: ~2-5ms
```

```
// - XOR 패킹: ~0.1ms
// - 총 처리 시간: ~2-5ms
```

병렬 처리

```
pub fn capacity_pack_range_with_data(
data: &mut [ChunkBytes],
mining_address: Address,
chunk_offset: std::ffi::c_ulong,
partition_hash: PartitionHash,
iterations: Option<u32>,
chunk_size: usize,
entropy_packing_iterations: u32,
irys_chain_id: u64,
) {
let iterations: u32 = iterations.unwrap_or(entropy_packing_iterations);
let mut entropy_chunk = Vec::<u8>::with_capacity(chunk_size);
data.iter_mut().enumerate().for_each(|(pos, chunk)| {
capacity_single::compute_entropy_chunk(
mining_address,
chunk_offset + pos as u64,
partition_hash.0,
iterations,
```

```
chunk_size,
&mut entropy_chunk,
irys_chain_id,
);
xor_vec_u8_arrays_in_place(chunk, &entropy_chunk);
});
}
```

검증 프로세스

패킹 검증

```
#[cfg(test)]
fn test_chunk_packing_unpacking() {
let mining_address = Address::random();
let chunk_offset = 1000;
let partition_hash = [0_u8; SHA_HASH_SIZE];
let chunk_size = 32;
let iterations = 2 * chunk_size as u32;

// 1. 엔트로피 생성
let mut entropy_chunk: Vec<u8> = Vec::with_capacity(chunk_size);
capacity_single::compute_entropy_chunk(
```

```
mining_address,
chunk_offset,
partition_hash,
iterations,
chunk_size,
&mut entropy_chunk,
testing_config.chain_id,
);

// 2. 테스트 데이터 생성
let data_size = chunk_size - 5; // 부분 청크 시뮬레이션
let mut data_bytes = vec![42_u8; data_size];

// 3. 패킹
let packed_data = packing_xor_vec_u8(entropy_chunk.clone(), &data_bytes);

// 4. PackedChunk 생성
let packed_chunk = PackedChunk {
data_root: H256::zero(),
data_size: data_size as u64,
data_path: Base64(vec![]),
bytes: Base64(packed_data),
tx_offset: TxChunkOffset::from(0),
```

```
packing_address: mining_address,
partition_offset: PartitionChunkOffset::from(chunk_offset  as u32),
partition_hash: H256::from(partition_hash),
};


// 5. 언패킹 및 검증
let  unpacked_chunk = unpack(&packed_chunk, iterations, chunk_size, testing_conf
assert_eq!(unpacked_chunk.bytes.0, data_bytes);
}
```

무결성 검증

1. **엔트로피 일관성**: 동일한 파라미터로 동일한 엔트로피 생성 확인
2. **가역성**: 패킹 후 언패킹하여 원본 데이터 복원 확인
3. **크기 검증**: 마지막 청크의 크기 조정 정확성 확인

보안 특성

암호학적 안전성

1. **일방향성**: 패킹된 데이터에서 엔트로피 역추적 불가능
2. **결정론적**: 동일한 입력에 대해 항상 동일한 결과
3. **확산성**: 입력의 작은 변화가 출력 전체에 영향
4. **충돌 저항성**: SHA-256의 충돌 저항성 상속

공격 저항성

1. **무차별 대입 공격**: 2^256의 키 공간으로 실질적으로 불가능
2. **패턴 분석 공격**: 2D 패킹으로 패턴 은폐
3. **사이드 채널 공격**: 상수 시간 연산으로 완화

최적화 기법

메모리 최적화

```
// 사전 할당된 버퍼 재사용
let mut entropy_chunk = Vec::<u8>::with_capacity(chunk_size);

// 불필요한 복사 방지
xor_vec_u8_arrays_in_place(chunk, &entropy_chunk);
```

캐시 효율성

```
// 순차적 메모리 접근 패턴
for i in 0..data.len() {
entropy[i] = entropy[i].bitxor(data[i]);
}
```

병렬화 전략

1. **청크 레벨 병렬화**: 독립적인 청크들을 병렬 처리
2. **파이프라인**: 엔트로피 생성과 XOR 연산 파이프라인

3. **SIMD 활용**: 벡터 연산으로 XOR 가속화

사용 예제

기본 패킹

```
use irys_packing::{capacity_pack_range_with_data, PackingType, PACKING_TYPE};


let mining_address = Address::random();
let chunk_offset = 0;
let partition_hash = PartitionHash::zero();
let chunk_size = 256 * 1024; // 256KB
let iterations = Some(2 * chunk_size as u32);


let mut chunks = vec![vec![0u8; chunk_size]; 4]; // 4개 청크


match PACKING_TYPE {
PackingType::CPU => {
capacity_pack_range_with_data(
&mut chunks,
mining_address,
chunk_offset,
partition_hash,
iterations,
```

```
chunk_size,
entropy_packing_iterations,
chain_id,
);
}
_ => unimplemented!(),
}
```

언패킹

```
use irys_packing::unpack;

let  unpacked_chunk = unpack(
&packed_chunk,
entropy_packing_iterations,
chunk_size,
chain_id,
);

println!("언패킹된 데이터 크기: {}", unpacked_chunk.bytes.0.len());
```

이 매트릭스 패킹 알고리즘은 Irys 체인의 데이터 보안과 저장 효율성을 보장하는 핵심 기술입니다.